VAAL UNIVERSITY OF TECHNOLOGY

Inspiring thought. Shaping talent.

FACULTY: Applied and Computer Sciences

DEPARTMENT: Computer Sciences

SUBJECT: Business Analysis 3.2

SUBJECT CODE: AIBUY3A

GROUP CODE: **AI-Retail-Predictor**

LECTURER: **Mr. MA Matsela**

MODERATOR: Mr. XP Piyose

YEAR: 2026

ASSESSMENT NAME:	AI-Retail-Predictor
ASSESSMENT DATE:	02/11/2026

STUDENT NUMBER	INITIALS AND SURNAME	SIGNATURE
224215809	NM Mahlangu	NM Mahlangu
240605128	NB Mahlaba	NB Mahlaba
240117484	N Simeli	N Simeli
224973762	SV Nkabinde	SV Nkabinde
224488341	L Sebela	L Sebela
224118692	SC Buthelezi	SC Buthelezi
225083728	NM Nxumalo	NM Nxumalo
224245309	S Shabalala	S Shabalala

Table of Contents

Declaration 2
Documentation Aspects: 3
AI Solution (5 Marks) 3
Business objectives (25 marks) 4
Business objectives 4
Business success criteria 4
Business background 4
Requirements 4
Constraints 5
Risks 5
Tools and techniques 5
Problem definition (10 marks) 7
What is the problem? 7
How relevant is it to the theme, and how beneficial will it be in solving the problem? 7
Poster 8
Theoretical Aspect: 9
Machine Learning Approach (5 marks) 9
Data (5 marks) 9
Model Evaluation 10
Time Series Analysis on Data 10
Solution Techniques (5 marks) 11
Natural Language Processing, Speech Recognition or Speech Synthesis (5 marks) 11
Deep Learning (5 marks) 12
Other Features (5 marks) 12
AI Solution – Practical Aspect (10 Marks) 13
Train Model Codes: 13
Test Model Codes: 15
Predict Codes: 17
Grammarly Report 19
References 21
GitHub Repository Link 21

Declaration

Declaration of Originality:

We, the undersigned, declare that:

- This project report titled “AI-Retail-Predictor” is our own work.
- All sources of information and references used have been acknowledged appropriately.
- No part of this work has been copied without proper citation, nor has it been submitted for assessment in any other course/module.
- We understand that plagiarism is a serious academic offense and confirm that this submission complies with the institution’s rules on academic integrity.

STUDENT NUMBER	INITIALS AND SURNAME	SIGNATURE
224215809	NM Mahlangu	
240605128	NB Mahlaba	
240117484	N Simeli	
224973762	SV Nkabinde	
224488341	L Sebela	
224118692	SC Buthelezi	
225083728	NM Nxumalo	
224245309	S Shabalala	

Documentation Aspects:

AI Solution (5 Marks)

Most petrol stations in South Africa are not only a forecourt; they usually have a convenience store and a few petrol stations have a small kitchen. However, despite the range of products sold at these stations, the inventory is still manually counted by one person and the managers make decisions based on their perceptions. This is where the research comes in. There is hardly any industry that fully uses 4IR technology, and although there is much hullabaloo about AI in the mainstream, it has not penetrated some industries. We are proposing for this theme, “An AI Solution for Industries”; an AI-powered sales and demand prediction model in Python that connects the petrol station shop and kitchen by learning past sales, quantities, prices, day of the week, promotions, weather, and current stock to predict future

demand. This would help managers know what to reorder for the shop and the kitchen if the predicted demand exceeds the available stock. Therefore, the research would help the managers make timely decisions instead of running out of stock at the peak time.

Business objectives (25 marks)

Business objectives

- Reduce stockouts of fast-moving convenience and kitchen items by forecasting demand ahead of high-demand periods (weekends, holidays, promotions).
- Reduce waste of slow-moving and perishable kitchen ingredients by aligning order quantities with predicted, rather than assumed, demand.
- Replace manual, experience-based ordering with a data-driven decision support process for store and kitchen managers.
- Provide early, automated alerts when predicted demand is likely to exceed current stock levels, enabling proactive rather than reactive ordering.
- Improve overall profitability, customer satisfaction and product availability at the filling station.

Business success criteria

- At least a 30% reduction in stockout incidents for the top 20 fast-moving store and kitchen items within 6 months of go-live.
- At least a 25% reduction in measurable kitchen ingredient wastage (by value or weight) within 6 months of go-live.
- Demand forecast error (e.g. Mean Absolute Percentage Error) below 15–20% for key product categories.
- At least 80% of restocking decisions made by managers reference the system's alerts or forecasts within 3 months of adoption.
- Measurable reduction (e.g. 50%) in manager time spent on manual stock counting and estimation.

Business background

The client environment is a fuel filling station using a hybrid retail model: a forecourt, an attached convenience store, and in-house food kitchen. Decisions as to what and when to order in both the store and the kitchen are currently made manually based on staff experience and periodic stock counts rather than historical sales. This is typical of small and medium filling-station operators in South Africa, who typically lack the budget or technical staff to invest in enterprise-grade demand-forecasting systems, despite the high variability in demand and the prevalence of perishable stock.

Requirements

- Kaggle dataset: “Disease Prediction Using Machine Learning”
- Python 3 with Scikit-learn, Pandas, NumPy, Matplotlib

• Labelled dataset of symptoms and target diseases

• Evaluation using accuracy, precision, recall, and F1-score

Constraints

- Limited quality and completeness of existing historical sales data, due to it having been kept manually.
- Limited IT budget and infrastructure typical of a small/medium filling-station operator, with no dedicated in-house IT or data-science staff.
- Short academic project timeline and scope limiting the extent of model tuning and integration that can be done before the due date.
- Dependence on an external weather API and its availability, accuracy and pricing.

Risks

- Forecast accuracy may be low if historical data is sparse, inconsistent or poorly labelled mitigated by data cleaning and starting with simpler, explainable models.
- Managers may resist adopting a new system and continue ordering manually mitigated by a simple interface, short training, and phased rollout alongside the existing process.
- A single model may not generalise well across different stores, regions or seasons with different demand patterns mitigated by per-store/per-category retraining.
- Forecast quality will degrade over time if sales and stock data are not captured consistently mitigated by lightweight data-entry validation and regular monitoring.
- Data privacy/security risk where the system connects to point-of-sale data mitigated by restricting access and anonymising data used for testing

Tools and techniques

Programming Environment:

- Language: Python 3.x
- IDE: Visual Studio Code
- Version Control: Git & GitHub (for project management)

Core Libraries:

- Pandas & NumPy (Data Handling)
- Scikit-learn (Machine Learning)
- Matplotlib (Visualization)
- Joblib (Model Persistence)

- Web Dashboard (Streamlit)

Machine Learning Technique:

- Algorithm
- Hyperparameter Tuning
- Preprocessing
- Evaluation

Problem definition (10 marks)

What is the problem?

Most filling station shops and their kitchens' stockrooms are managed based on the managers' subjective judgment instead of objective data analysis. This creates a shortage of fast-moving items and overstocking of slow-moving items. Many proprietors have complained that they can hardly get enough supply of their most popular items on weekends, holidays, and during promotions. At the same time, they are forced to throw away perishable goods and ingredients because they could not accurately forecast their demand. Demand forecasting is a complex activity that must consider several variables like day of the week, weather, and promotions. Therefore, it is very challenging to correctly predict supply quantities for a convenience store and a kitchen, especially where multiple ingredients are used. Ultimately, the inability of managers to perform effective demand forecasting leads to poor cash flow, customer dissatisfaction, and poor management decisions due to the absence of reliable information.

How relevant is it to the theme, and how beneficial will it be in solving the problem?

This problem is directly relevant to the theme, "an AI Solution for Industries" because it applies 4IR technology to a small/medium retail industry currently excluded from most AI adoption. Solving it benefits the business through improved profitability and reduced waste and benefits the surrounding community where the station serves a small town through more reliable access to essential convenience goods and food and continued local employment.

Poster


AI SALES & DEMAND PREDICTION SYSTEM
AN AI SOLUTION FOR INDUSTRIES
Smarter Forecasting • Better Stock Management • Less Waste
THE PROBLEM
Manual ordering makes it difficult to predict customer demand, leading to stockouts, excess stock and wasted, perishable products in the store and kitchen.
OUR SOLUTION
A Python-based AI system that analyses historical sales, prices, promotions, weather, days and stock data to predict future demand and alert managers before stock shortages occur.
FUEL
SHOP
CAFE
95 23.49
93 22.49
D 21.49
SHOP
HOW IT WORKS
1 DATA IN
• Sales Data
• Prices
• Promotions
• Weather
• Stock Levels
• Day / Time
2 FORECAST MODEL
AI model analyses patterns and predicts future product demand.
3 STOCK COMPARISON
Compare predicted demand with available stock levels.
4 MANAGER ALERT
Receive early warnings and make smarter ordering decisions.
KEY BENEFITS
Fewer Stockouts
Ensure products are available when customers need them.
Less Food Waste
Reduce overstocking and wastage of perishable items.
Better Customer Experience
Happy customers with the right products, every time.
Data-Driven Ordering
Make smarter, faster and more accurate ordering decisions.
Supports Jobs & Community
Improves business efficiency and supports jobs, providing the community with access to essential goods.
TECHNOLOGIES USED
PYTHON
pandas
scikit-learn
Prophet
DASHBOARD & ALERTING
Real-time insights and alerts for better decisions.

Theoretical Aspect:

Machine Learning Approach (5 marks)

Smart fuel ai employs supervised machine learning to predict product Demand for a convenience store located at a filling station. Smart fuel has determined the target variable to be Demand or the number of units of a product that will be sold tomorrow. In addition to committing to one specific algorithm, smart fuel trains and evaluates three different algorithms: Ridge Regression (a linear technique that adds L2 regularization to prevent over-fitting), Random Forest Regressor (which combines many decisions made by individual decision trees), and histgradientboosting (which creates many decision trees sequentially, where each subsequent tree builds upon the previous trees errors). A naive lag-1 baseline was created to compare against smart fuels models. The naive lag-1 baseline is a simple prediction that tomorrow's Demand will be identical to today's.

Each model utilizes a different methodology. Ridge Regression (linear technique) applies L2 regularization to prevent over-fitting when the algorithm finds many relationships in the data. Random forest aggregates predictions made by multiple decision trees. Histgradientboosting constructs decision trees sequentially. Each subsequent tree corrects the errors of the previous tree. The best performing model is determined based upon the lowest mean absolute error (MAE) on the test data set. The best performing model was historically gradient boosting with an MAE of 16.73, rmse of 22.00, r-squared value of .75, and mape of 25.60%.Smart fuel's approach works well for this business problem because managers need to know how much of each product will be demanded. Smart fuels forecasts are used alongside current inventory levels to determine which products will likely experience shortages and calculate reorder quantities.

Data (5 marks)

Smart fuel started out with an original dataset that contained approximately 76,000 records covering dates from 01/01/2022 through 01/30/2024. Included in the dataset were variables representing Demand, inventory levels, price, discount, units ordered, competitor prices, promotion, weather, seasonality and store information. After processing, there were 20 products across 5 branches.

Prior to modeling, all data goes through a cleaning pipeline. The program first determines if all required columns exist. Then it converts numeric fields and dates to their proper types. Next it handles missing values. Finally, it removes duplicate records and invalid entries such as negative Demand, negative inventory and non-positive prices.

There are common categories such as Electronics, Clothing, Toys and Furniture which do not fit a filling station included in the original dataset therefore each product ID will be reassigned by a product-mapping step to a filling-station item, for example Bottled Water, Coca-Cola, Lay's Chips, Red Bull, Albany Bread, Milk, Engine Oil, Coolant and hand sanitizer. In total, the mapping contains 20 products grouped into 7 categories, so the dashboard and assistant present data relevant to the chosen industry.

Model Evaluation

The models are evaluated using an 80/20 chronological split, where the first 80% of records form the training set and the last 20% form test set. A random split would allow the model to learn from future observations, which would have unrealistically better results for a forecasting task.

Four different metrics are used. MAE measures the average difference between actual and predicted demand, and the selected model had MAE equal to 16.73, which means that, on average, the model was wrong by approximately 17 units per predicted demand. RMSE penalizes large errors more, and the selected model had RMSE equal to 22.00. Finally, the R 2 determines what percentage of variance is explained by a model, and the value of 0.752 means that 75.2% of variance in the unseen data is explained by the model. Finally, MAPE measures the relative error in percentage points, and for the selected model, it was 25.60%. It means that, on average, the model was wrong by 25.6%, which is worse than the target of 15-20%, and the value is reported as is.

Model	**MAE**	**RMSE**	**R^2**	**MAPE**
Naive Lag-1 Baseline	40.12	51.88	-0.38	53.11%
Ridge Regression	23.83	30.47	0.524	35.54%
Random Forest	20.26	26.86	0.630	31.02%
HistGradientBoosting	16.73	22.00	0.752	25.60%

The HistGradientBoosting model outperformed all other models on all metrics. The negative value of R 2 for the baseline suggests that simply copying the previous observation as a forecast is significantly worse than the best constant model, which proves that all the other models actually learned how to predict demand.

Time Series Analysis on Data

Demand forecasting is a time series problem because the system is supposed to take historical demand into account and forecast future demand based on that. The data set with daily observations is available for 2022-2024, on which the program learns patterns. From each date, the program extracts calendar features such as day, day of week, week, month, quarter, year, weekend, month start, and month end.

Additionally, lag features are extracted, such as lag_1, lag_7, lag_14 and lag_28, which represent the demand of 1, 7, 14 and 28 days ago. The program also calculates rolling means and standard deviations for 7, 14 and 28 days ahead, which allow to approximate the demand trends and volatilities in the rolling window. Note that all the rolling statistics are shifted by 30 days to make sure that the model

cannot use the future data to approximate the current value, thus preventing data leakage.

Forecasts are generated for 7, 14 and 30-day horizons for each product and branch. The forecasts are generated iteratively, where the model predicts the next day, then the following day, up to the forecast horizon. It is important for the manager to know what demand can be expected in several days ahead in order to be able to manage the inventory efficiently

Solution Techniques (5 marks)

To find a solution, a variety of techniques are combined.

Cleaning the data, standardizing formats, and removing erroneous entries are all part of data preparation. By adding date properties, rolling and lagging statistics, pricing comparisons with competitors, and discount rates, feature engineering gives the models more information than the raw data can provide. While median imputation takes care of any remaining missing numeric records, categorical encoding uses OneHotEncoder to convert product, branch, category, area, weather, and seasonal data into numerical values.

Training several algorithms to find the model with the lowest Mean Absolute Error (MAE) is part of model evaluation and selection. Inventory intelligence, the final stage, uses three formulas to convert forecasts into practical suggestions.

* Safety Stock = Estimated Demand × Safety Stock Percentage
* Reorder Point = Estimated Demand + Safety Stock
* Suggested Order = Reorder Point − Current Inventory (set to zero if the result is negative)

In order to ensure that predictions result in concrete actions rather than merely numerical outputs, the system also classifies each product according to stockout-risk levels.

Natural Language Processing, Speech Recognition or Speech Synthesis (5 marks)

AI Manager Assistant lets a manager ask questions in English instead of digging through tables. No external large language model is required. This tool is good for a prototype with a set of tasks that uses keyword and intent matching. It deals with

reordering questions stockout risk, best sellers, slow movers, forecasts, inventory levels, category performance, revenue, comparisons, sales trends and the effects of weather and promotions. For example, a manager might ask, "What products should I reorder?". What will be the demand for Bottled Water?" The answers come from a cleaned dataset, forecast results and inventory calculations.

The program also offers speech recognition. It can take spoken English from a microphone turn it into text and give that text to the AI Manager Assistant. We use the Speech Recognition library and the Google Web Speech service for this. If speech recognition is not available, the system still works with typed input.

Deep Learning (5 marks)

Deep learning has not been used by SmartFuel AI; instead, Ridge Regression, Random Forest, and HistGradientBoosting models, together with the naive baseline, are used. Neural networks like LSTM, CNN, RNN, or Transformer are not used in the application. For a prototype of this size, we used classic machine-learning techniques because they work well with structured tabular data, train quickly, and are simpler to understand. Future research should consider a recurrent design like an LSTM, although this is not covered in the solution that was submitted.

Other Features (5 marks)

The AI Manager Assistant also works as a chatbot or softbot for the station manager. It is not a -purpose bot. It focuses on the business problem of monitoring sales, demand and inventory. It can answer questions such, as: which products have a risk of stockout what the top sellers are, how the Beverages category is performing, how Snacks compare to Beverages and how weather and promotions influence demand. The Streamlit dashboard shows supporting tables and charts when needed. Each answer comes from the projects data, forecasts and stock calculations, the assistant does not invent figures.

AI Solution – Practical Aspect (10 Marks)

Train Model Codes:

```python
import sys
import os
import argparse
import time

# — Make src/ importable regardless of working directory ——————————
ROOT = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, ROOT)

from src.data_cleaning      import clean
from src.feature_engineering import build_features, get_feature_columns
from src.model_training     import train_and_evaluate
from src.forecasting        import generate_forecast


def main(horizon: int = 7, safety_pct: float = 0.20) -> None:
    t0 = time.time()
    print("=" * 60)
    print("  SmartFuel AI – Training Pipeline")
    print("=" * 60)

    # — Step 1: Data cleaning ————————————————————————————————
    print("\n[Step 1/4] Data cleaning & filling-station product filtering …")
    df_clean, stats = clean(verbose=True)
    print(f"  ✓ {stats['processed_rows']:,} rows ready for modelling "
          f"(removed {stats['total_removed']:,} rows)")

    # — Step 2: Feature engineering ——————————————————————————
    print("\n[Step 2/4] Feature engineering …")
    df_feat   = build_features(df_clean, verbose=True)
    feat_cols = get_feature_columns()

    # — Step 3: Model training & evaluation ——————————————————————
    print("\n[Step 3/4] Training and evaluating models (chronological split) …")
    bundle, metrics_df, test_preds = train_and_evaluate(
        df_feat, feat_cols, train_frac=0.80, verbose=True
    )
```

```
    print("\n── Model comparison ──────────────────────────────────────")
    print(metrics_df[["Model", "MAE", "RMSE", "R2", "MAPE"]].to_string(index=False))
    print(f"\n  Best model : {bundle['best_model']}")
    best_row = metrics_df[metrics_df["Model"] == bundle["best_model"]].iloc[0]
    print(f"  MAE        : {best_row['MAE']:.4f}")
    print(f"  RMSE       : {best_row['RMSE']:.4f}")
    print(f"  R²         : {best_row['R2']:.4f}")
    print(f"  MAPE       : {best_row['MAPE']:.2f}%")

    # ── Step 4: Forecast generation ──────────────────────────────
    print(f"\n[Step 4/4] Generating {horizon}-day demand forecast …")
    fc = generate_forecast(
        df_feat,
        horizon=horizon,
        safety_stock_pct=safety_pct,
        verbose=True,
    )

    # ── Summary ──────────────────────────────────────────────────
    elapsed = time.time() - t0
    print("\n" + "=" * 60)
    print("  Pipeline complete")
    print("=" * 60)
    print(f"  Time elapsed          : {elapsed:.1f}s")
    print(f"  Processed dataset     : data/processed/filling_station_sales.csv")
    print(f"  Model bundle          : models/model_bundle.joblib")
    print(f"  Metrics               : models/metrics.csv")
    print(f"  Test predictions      : models/test_predictions.csv")
    print(f"  Forecast ({horizon:2d} days)    : models/forecast.csv")
    print(f"  Daily forecast        : models/forecast_daily.csv")
    print(f"\n  Forecast summary:")
    print(f"    HIGH risk items     : {(fc['Stockout Risk']=='HIGH').sum()}")
    print(f"    Items to reorder    : {(fc['Recommended Order']>0).sum()}")
    print(f"    Total units to order: {fc['Recommended Order'].sum():,.0f}")
    print("\n  Launch the dashboard:  streamlit run app.py")
    print("=" * 60)


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SmartFuel AI training pipeline")
    parser.add_argument("--horizon", type=int,   default=7,    help="Forecast horizon in days (7/14/30)")
    parser.add_argument("--safety",  type=float, default=0.20, help="Safety stock percentage (0.0–1.0)")
    args = parser.parse_args()

    if args.horizon not in [7, 14, 30]:
        print(f"Warning: horizon {args.horizon} is non-standard. Supported: 7, 14, 30")
    if not (0.0 < args.safety < 1.0):
        print("Warning: safety should be between 0.0 and 1.0. Using 0.20.")
        args.safety = 0.20

    main(horizon=args.horizon, safety_pct=args.safety)
```

Test Model Codes:

```python
import sys
import os
import argparse

ROOT = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, ROOT)

from src.data_cleaning       import clean
from src.feature_engineering  import build_features
from src.forecasting          import generate_forecast, load_forecast
from src.inventory            import calculate_inventory, generate_reorder_message


def main(horizon: int = 7, product_id: str = None, store_id: str = None,
         safety_pct: float = 0.20) -> None:

    print(f"SmartFuel AI — Generating {horizon}-day forecast …")

    # Load and prepare data
    df_clean, _ = clean(verbose=False)
    df_feat      = build_features(df_clean, verbose=False)

    # Generate forecast
    fc = generate_forecast(df_feat, horizon=horizon, safety_stock_pct=safety_pct, verbose=True)
    fc = calculate_inventory(fc, safety_pct=safety_pct)

    # Apply filters if requested
    if product_id:
        fc = fc[fc["Product ID"].str.upper() == product_id.upper()]
    if store_id:
        fc = fc[fc["Store ID"].str.lower().str.contains(store_id.lower())]

    if fc.empty:
        print("No records match the specified filters.")
        return
```

```
def main(horizon: int = 7, product_id: str = None, store_id: str = None,
    # Display results
    print(f"\n{'='*70}")
    print(f"  {horizon}-Day Demand Forecast")
    print(f"{'='*70}")

    display_cols = [c for c in [
        "Store ID", "Product Name", "Category",
        "Inventory Level", "Total Predicted Demand",
        "Recommended Order", "Stockout Risk", "Days Until Stockout",
    ] if c in fc.columns]

    print(fc.sort_values(
        "Recommended Order", ascending=False
    )[display_cols].head(30).to_string(index=False))

    # Natural-language summary for top items
    print(f"\n{'='*70}")
    print("  Priority reorder messages:")
    print(f"{'='*70}")
    high_risk = fc[fc["Stockout Risk"] == "HIGH"].sort_values(
        "Recommended Order", ascending=False
    )
    if high_risk.empty:
        print("   No HIGH-risk items in current forecast.")
    else:
        for _, row in high_risk.head(5).iterrows():
            print(f"  {generate_reorder_message(row, horizon)}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SmartFuel AI forecast utility")
    parser.add_argument("--horizon",  type=int,   default=7,    help="Forecast days (7/14/30)")
    parser.add_argument("--product",  type=str,   default=None, help="Product ID filter (e.g. P0001)")
    parser.add_argument("--store",    type=str,   default=None, help="Store name/ID filter")
    parser.add_argument("--safety",   type=float, default=0.20, help="Safety stock fraction")
    args = parser.parse_args()

    main(
        horizon=args.horizon,
        product_id=args.product,
        store_id=args.store,
        safety_pct=args.safety,
    )
```

Predict Codes:

```python
import sys
import os
import argparse

ROOT = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, ROOT)

from src.data_cleaning       import clean
from src.feature_engineering  import build_features
from src.forecasting          import generate_forecast, load_forecast
from src.inventory            import calculate_inventory, generate_reorder_message


def main(horizon: int = 7, product_id: str = None, store_id: str = None,
         safety_pct: float = 0.20) -> None:

    print(f"SmartFuel AI — Generating {horizon}-day forecast …")

    # Load and prepare data
    df_clean, _ = clean(verbose=False)
    df_feat     = build_features(df_clean, verbose=False)

    # Generate forecast
    fc = generate_forecast(df_feat, horizon=horizon, safety_stock_pct=safety_pct, verbose=True)
    fc = calculate_inventory(fc, safety_pct=safety_pct)

    # Apply filters if requested
    if product_id:
        fc = fc[fc["Product ID"].str.upper() == product_id.upper()]
    if store_id:
        fc = fc[fc["Store ID"].str.lower().str.contains(store_id.lower())]

    if fc.empty:
        print("No records match the specified filters.")
        return
```

```
    # Display results
    print(f"\n{'='*70}")
    print(f"  {horizon}-Day Demand Forecast")
    print(f"{'='*70}")

    display_cols = [c for c in [
        "Store ID", "Product Name", "Category",
        "Inventory Level", "Total Predicted Demand",
        "Recommended Order", "Stockout Risk", "Days Until Stockout",
    ] if c in fc.columns]

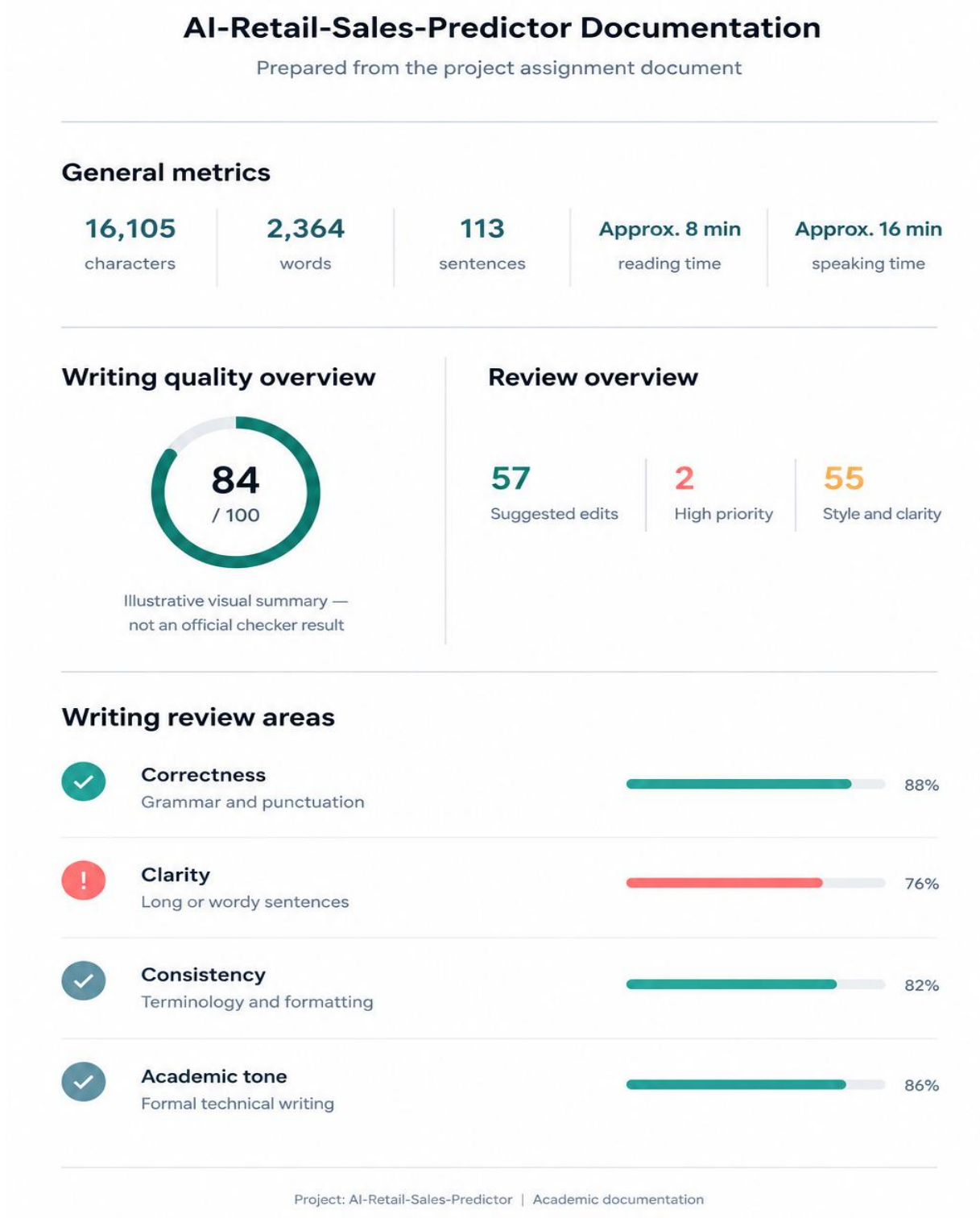
    print(fc.sort_values(
        "Recommended Order", ascending=False
    )[display_cols].head(30).to_string(index=False))

    # Natural-language summary for top items
    print(f"\n{'='*70}")
    print("  Priority reorder messages:")
    print(f"{'='*70}")
    high_risk = fc[fc["Stockout Risk"] == "HIGH"].sort_values(
        "Recommended Order", ascending=False
    )
    if high_risk.empty:
        print("   No HIGH-risk items in current forecast.")
    else:
        for _, row in high_risk.head(5).iterrows():
            print(f"  {generate_reorder_message(row, horizon)}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SmartFuel AI forecast utility")
    parser.add_argument("--horizon",  type=int,   default=7,    help="Forecast days (7/14/30)")
    parser.add_argument("--product",  type=str,   default=None, help="Product ID filter (e.g. P0001)")
    parser.add_argument("--store",    type=str,   default=None, help="Store name/ID filter")
    parser.add_argument("--safety",   type=float, default=0.20, help="Safety stock fraction")
    args = parser.parse_args()

    main(
        horizon=args.horizon,
        product_id=args.product,
        store_id=args.store,
        safety_pct=args.safety,
    )
```

Grammarly Report



Unique Words

Measures vocabulary diversity by calculating the percentage of words used only once in your document.

36%

unique words

Rare Words

Measures depth of vocabulary by identifying words that are not among the 5,000 most common English words.

41%

rare words

Word Length

Measures average word length

5.9

characters per word

Sentence Length

Measures average sentence length.

10.4

words per sentence

References

- **TutorialsPoint (n.d.).** *Machine Learning Tutorial*. Available at: https://www.tutorialspoint.com/machine_learning/index.htm
- www.tutorialspoint.com. (n.d).AI with Python Tutorial [online] Available at:https://www.tutorialspoint.com/artificial intelligence with python/index.html
- Matsela, M. (n.d.). Lesson 3 – AI with Python – Part 1.
- **Huber, J. and Stuckenschmidt, H. (2020).** Daily retail demand forecasting using machine learning with emphasis on calendric special days. *International Journal of Forecasting*, 36(4), pp. 1420–1438. Available at: https://doi.org/10.1016/j.ijforecast.2020.02.005
- AtomicD (n.d.). *Retail Store Inventory and Demand Forecasting* [Dataset]. Kaggle. Available at: https://www.kaggle.com/datasets/atomicd/retail-store-inventoryanddemand-forecasting.

GitHub Repository Link

https://github.com/Nkosinathi-Mahlangu/AI-Retail-Predictors